

PART 01 — Import Libraries & Load Dataset

Importing Libraries

The first step in any machine learning project is importing the required libraries. Different libraries perform different tasks throughout the project.

- **Pandas** is used for loading and manipulating datasets.
- **NumPy** is used for numerical computations.
- **Matplotlib** and **Seaborn** are used for data visualization.
- **Warnings** are ignored to keep the notebook output clean.
- **pd.set_option()** is used to display all dataset columns without truncation.

Finally, both the training and testing datasets are loaded into memory.

Why are we doing this?

Before building any model, we first need access to the dataset and the tools required to analyze and process it.

Think of this as gathering all the equipment before starting an experiment.

Interview Tip

Question: Why do we use Pandas?

Answer:

Pandas provides powerful data structures such as DataFrames, making it easy to load, clean, manipulate, analyze, and preprocess structured datasets efficiently.

PART 02 — Understanding the Dataset

Before cleaning or building a model, it is important to understand the dataset.

In this step, we inspect:

- Number of rows and columns
- Feature names
- Numerical and categorical variables
- Data types
- Missing values
- Basic statistical information

Understanding the dataset helps identify potential problems before preprocessing begins.

? Why?

Imagine a doctor giving medicine before checking the patient.

That wouldn't make sense.

Similarly,

We first **understand** the dataset,

Then we **clean** it.

Interview Tip

Question Why do we use `info()`?

Because it quickly tells us:

- Missing values
- Data types
- Number of non-null values
- Memory usage

Which helps us decide the preprocessing strategy.

PART 03 — Separate Numerical & Categorical Features

Feature Types

Machine learning preprocessing depends on the type of data.

Therefore, the dataset is divided into two groups:

- **Numerical Features:** Continuous or discrete numeric values.
- **Categorical Features:** Text-based values representing categories.

Since different preprocessing techniques are required for each type, separating them makes the pipeline cleaner and easier to manage.

Why?

Because Median, Mean, Scaling, and Normalization are only meaningful for numerical features.

Whereas encoding, Label Encoding, and One-Hot Encoding are only applied to categorical features.



Interview Tip

Question: Why do we separate numerical and categorical features?

Answer

Different feature types require different preprocessing techniques. Numerical features may need scaling or imputation, whereas categorical features require encoding before machine learning algorithms can use them.

PART 04 — Target Variable Analysis

The target variable for this project is SalePrice, the selling price of each house.

Before building a regression model, it is essential to understand the distribution of the target variable.

The histogram shows whether the data follows a normal distribution, while the boxplot helps identify potential outliers.

Additionally, skewness is calculated to measure how asymmetric the distribution is.

A highly positive skew indicates that a small number of expensive houses are stretching the distribution, which can negatively affect regression models.

? Why are we doing this?

This is one of the most important steps in any regression project.

A professional Data Scientist **always studies the target variable before training a model.**

Many regression algorithms perform better when the target distribution is closer to normal.

If the target is heavily skewed (as is the case in the House Prices dataset), a **log transformation** often improves model performance and reduces prediction errors.

We are not applying the log transformation yet; first, we confirm whether it is actually needed.

Interview Tip

Question: Why analyze the target variable before training?

Answer:

Understanding the target variable helps identify skewness, outliers, and distribution patterns. This information guides preprocessing decisions such as log transformation and helps improve the performance and stability of regression models.

PART 05 — Missing Value Analysis **(Professional EDA)**

Step 1 — Check Missing Values

Real-world datasets often contain incomplete information.

Before applying any imputation technique, it is important to identify which features contain missing values and how many values are missing in each feature.

This helps in selecting the most appropriate strategy instead of treating every feature the same way.

? Why?

Imagine a hospital dataset.

Age

Gender

Blood Pressure

Disease

If **Age** is missing, we might replace it using the median.

But if **GarageType** is missing in a house dataset, it probably means:

The house doesn't have a garage.

Replacing it with the median would make no sense.

Different features require different treatments.



Interview Tip

Q: Why shouldn't we fill every missing value with the median?

Answer:

Because missing values have different meanings depending on the feature. Numerical features, categorical features, and features where missing indicate the absence of something and should be handled differently.

Step 2 - Missing Percentage

The absolute number of missing values is not always enough to judge data quality.

Calculating the percentage of missing values provides a clearer understanding of how much information is absent in each feature.

Features with extremely high missing percentages may contribute little useful information and can be considered for removal.

Why?

Imagine two columns:

Column A → 5 missing values

Column B → 500 missing values

Without knowing the dataset size, we cannot judge which one is problematic.

If the dataset contains 10,000 rows:

5 missing values = only 0.05%

500 missing values = 5%

Percentages provide better context.

Step 3 — Create Missing Value Table

A structured summary table provides a complete overview of missing values within the dataset.

It displays both the total number of missing values and the percentage of missing data for every affected feature.

This table makes it easier to prioritize preprocessing decisions.

Interview Tip

Recruiters love seeing summary tables.

Instead of saying

"There were missing values."

You can say

I created a missing value summary table to quantify the amount of missing information before selecting feature-specific imputation strategies.

Step 4 — Visualize Missing Values

A heatmap provides a visual representation of missing values across the dataset.

Each colored cell indicates a missing observation, allowing us to quickly identify patterns and determine whether missing values occur randomly or follow a specific structure.

Why?

Numbers tell us how many values are missing.

A heatmap tells us where they are missing.

Sometimes an entire section of the dataset is missing together.

That indicates a pattern.

Patterns are valuable.

Random missing values and structured missing values are treated differently.

Step 5 — Which Features Need Attention?

Now we'll inspect the most problematic features.

For example, you may see something like:

Feature	Missing %
PoolQC	99%
MiscFeature	96%
Alley	93%
Fence	81%
FireplaceQu	47%

Observations

Several features contain a high percentage of missing values.

However, missing values in this dataset often have a specific meaning.

For example:

- Missing PoolQC usually indicates that the house does not have a swimming pool.
- Missing GarageType often means the house has no garage.
- Missing Fence may indicate the absence of a fence.

Therefore, these missing values should not automatically be treated as errors.

 Professional Insight

This is where many beginners make a mistake.

They write:

```
train.fillna(train.median())
```

or

```
train.fillna("None")
```

for the entire dataset.

It works...

But it's not the best approach.

A professional asks:

Why is this value missing?

Sometimes, missing itself is information.

For example:

- PoolQC = NaN → No pool.
- GarageType = NaN → No garage.

Replacing these with "None" is meaningful because it preserves the fact that the feature is absent, rather than inventing a value.

PART 06 — Data Cleaning & Missing Value Handling

Step 1 — Make Copies of Dataset

Before modifying the data, copies of the original datasets are created. This preserves the raw data, allowing experiments and preprocessing steps to be performed safely without permanently changing the original dataset.

Why?

Imagine after 2 hours you realize you made a mistake.

Without a copy...

You'll have to reload everything.

Professional Data Scientists almost always create a working copy.

Interview Tip

Q: Why use `.copy()` instead of modifying the original DataFrame?

Creating a copy preserves the original dataset, making the workflow reproducible and preventing accidental modifications during preprocessing.

Step 2 — Drop Features with Extremely High Missing Values

Removing Features with Excessive Missing Values

Some features contain more than 80% missing values, meaning only a very small number of houses actually have information recorded.

Such features contribute little useful information while increasing data complexity.

Therefore, these columns are removed from both the training and testing datasets.

? Why?

Suppose a feature has

95% missing values.

Only

5%

of houses actually contain useful information.

Instead of helping,

it mostly introduces noise.



Interview Tip

Q: When should a feature be dropped?

When a feature contains an extremely high percentage of missing values and contributes little predictive value, removing it can simplify the model and reduce noise.

Step 3 — Handle Numerical Missing

ValuesMissing values in numerical features are replaced using the median of each feature.

The median is preferred because it is less sensitive to extreme values and outliers than the mean, making it a more robust choice for real-world datasets.

❓ Why Median?

Example

House Prices

100000

120000

140000

170000

5,000,000

Mean becomes huge because of one luxury house.

Median stays realistic.

Interview Tip

Q: Why median instead of mean?

Answer

The median is resistant to outliers, making it a better representation of the central tendency for skewed numerical data.

Step 4 — Handle Categorical Missing Values

Missing values in categorical features are replaced with "None".

In this dataset, missing categorical values often indicate the absence of a feature (such as no garage, no fireplace, or no basement) rather than unknown information.

Replacing them with "None" preserves this meaning.

❓ Why not Mode?

Suppose, GarageType

Attached

Detached

NaN

If we fill in the attached, we incorrectly tell the model. This house has an attached garage.

Reality: It has no Garage. Those are very different.

Step 5 — Verify Missing Values

After preprocessing, the datasets are checked again to confirm that all missing values have been handled successfully.

Ensuring a complete dataset prevents errors during feature encoding and model training.

 **Interview Tip**

Always verify your preprocessing.

Never assume it worked.

Recruiters love seeing validation steps.

Step 6 — Duplicate Records

Duplicate records are examined to ensure data quality.

Duplicate observations can bias the learning process by giving certain samples more influence than others.

If duplicates are present, they are removed before model training.

Step 7 — Final Dataset Check

After cleaning, the datasets are inspected again to verify:

- Missing values have been removed.
- Unnecessary features have been dropped.

- Data types remain correct.
- The datasets are ready for feature engineering and preprocessing.

PART 07 — Exploratory Data Analysis (EDA)

Step 1 — Correlation Analysis

Correlation measures the strength and direction of the relationship between two numerical variables.

A value close to:

- $+1 \rightarrow$ Strong Positive Relationship
- $0 \rightarrow$ No Relationship
- $-1 \rightarrow$ Strong Negative Relationship

Understanding feature correlation helps identify the variables that have the greatest influence on house prices.

 **Why?**

Suppose

OverallQual $\rightarrow$ 0.82

This means

Better quality houses generally have higher prices.

Now suppose

BedroomAbvGr $\rightarrow$ 0.15

Bedrooms don't affect price nearly as much.

That's useful information.



Interview Tip

Q: Why calculate correlation?

Answer:

Correlation helps identify which numerical features are most strongly related to the target variable, making it easier to understand the data and guide feature engineering or selection.

Step 2 — Top 15 Correlated Features

The strongest positively correlated features are displayed to identify the variables that contribute most to predicting house prices.

These features are often the most informative for regression models.

Step 3 — Correlation Heatmap

A correlation heatmap provides a visual summary of relationships among numerical features.

Features with stronger positive or negative relationships appear more prominently, making it easier to identify useful predictors and highly correlated variables.

 **Why?**

Instead of reading

200 correlation numbers,

our brain understands colours much faster.

That's why visualization is important.

Step 4 — Top 10 Features with SalePrice

This chart highlights the ten numerical features most strongly associated with house prices.

Understanding these relationships provides valuable business insight and helps identify influential variables for predictive modeling.

Step 5 — Scatter Plot (Most Important Feature)

The scatter plot visualizes the relationship between ground living area and house prices.

An upward trend indicates that houses with larger living areas generally sell for higher prices.

 **Why?**

Regression models learn relationships.

Scatter plots let us visually verify whether such relationships actually exist.

Step 6 — Overall Quality vs SalePrice

The boxplot compares house prices across different overall quality ratings.

Higher quality ratings generally correspond to higher selling prices, indicating that construction quality is an important predictor.

Interview Tip

Q: Why use a boxplot instead of a histogram?

Answer:

A boxplot compares the distribution of a numerical variable across different categories while also highlighting the median, spread, and potential outliers.

Step 7 — Distribution of Important Numerical Features

Histograms are used to understand the distribution of important numerical variables.

This helps identify skewness, unusual patterns, and potential preprocessing requirements such as transformations.

Step 8 — Outlier Detection

Outliers are observations that differ significantly from the majority of the data.

They are identified using boxplots to determine whether extreme values require further investigation.

In this project, outliers are examined but not removed immediately, as some may represent genuine high-value properties.

? Why don't we remove outliers immediately?

Because not every outlier is a mistake.

Imagine a luxury villa worth \$1,000,000.

It's an outlier...

But it's a real house.

Deleting it would reduce the model's ability to predict expensive homes accurately.

Professional practice is to **investigate first, remove only if justified.**

PART 08 — Feature Engineering & Data Preprocessing

Step 1 — Create New Features

A new feature called HouseAge is created by subtracting the year the house was built from the year it was sold.

Instead of allowing the model to learn from two separate columns (YearBuilt and YrSold), this feature directly represents the age of the property, making the relationship easier to understand.

Why?

Compare these two houses:

Year Built	Year Sold
2008	2010
1950	2010

The second house is much older.

Instead of giving the model two dates,
we directly tell it

This house is 60 years old.

That's much easier for the model.

Step 2 — Remodel Age

The age since the last remodeling is calculated. Recently renovated houses often have higher market value than houses that have not been renovated for many years.

Step 3 — Total Bathrooms

A combined bathroom feature is created by counting full bathrooms as 1 and half bathrooms as 0.5.

This provides a more realistic measure of the total bathroom capacity of the house.

Why?

Instead of learning from

FullBath, HalfBath, BsmtFullBath, BsmtHalfBath

The model learns TotalBathrooms

Much simpler.

Step 4 — Total Living Area

Total Square Footage

Instead of treating the basement, first floor, and second floor separately, a new feature called TotalSF is created.

This represents the total usable indoor area of the house.

Why?

A buyer doesn't care where the area is.

They care about

Total space.

Step 5 — Total Porch Area

All porch-related features are combined into a single feature representing the total outdoor porch area.

This reduces feature redundancy and captures the complete outdoor living space.

Step 6 — Check Newly Created Features

The newly engineered features are inspected to verify that they have been created correctly before further preprocessing.

Step 7 — Target Variable Log Transformation

Log Transformation of Target Variable

The target variable (SalePrice) is transformed using the natural logarithm ($\log_{1p}$).

House prices are positively skewed, meaning a small number of very expensive houses stretch the distribution.

Applying a logarithmic transformation reduces skewness, stabilizes variance, and often

improves the performance of regression models.

Why?

Earlier, we calculated skewness. Suppose it was 1.88 That's highly skewed.

Regression prefers something closer to 0. So we transform it.

Why $\log_{1p}()$ instead of $\log()$?

Because $\log(0)$ doesn't exist.

But $\log_{1p}(0)$

$\log(1)$

0

Safer.



Interview Tip

Q: Why apply log transformation only to SalePrice?

Because the target variable is highly right-skewed. Log transformation reduces skewness, stabilizes variance, and improves regression performance. Since SalePrice is what the model predicts, transforming it often leads to more accurate predictions.

Step 8 — One-Hot Encoding

Categorical variables are converted into numerical form using One-Hot Encoding.

Machine learning models cannot directly process text values, so each category is transformed into a binary indicator column.

Step 9 — Align Train & Test

After encoding, the training and testing datasets may contain different columns because some categories exist only in one dataset.

The datasets are aligned to ensure both contain the same feature set before model training and prediction.

Missing columns are filled with zeros.

Why?

Imagine a train has blue, green, and red.

The test has blue and green. The model expects 3 columns. But the test has 2 columns.

Prediction fails. Alignment fixes this automatically.

Step 10 — Verify Shapes

The final shapes of the training and testing datasets are verified to ensure both contain compatible features after preprocessing.

PART 09 — Model Building

Step 1 — Separate Features and Target

The dataset is divided into:

- Features (X) → Independent variables used for prediction.
- Target (y) → House prices that the model will learn to predict.

This separation is required before training any supervised learning model.

Step 2 — Train Validation Split

The dataset is divided into:

- 80% Training Data
- 20% Validation Data

The training data is used for learning, while the validation data is used to evaluate how well the model performs on unseen data.

A fixed `random_state` is used to ensure reproducible results.

Step 3 — Check Shapes

The dimensions of the training and validation datasets are checked to ensure the split has been performed correctly.

Step 4 — Import Linear Regression

Linear Regression is selected as the baseline model because it is simple, interpretable, and provides a good benchmark for comparing more advanced regression algorithms.

Step 5 — Create Model

A Linear Regression model is initialized with default parameters.

At this stage, the model has not yet learned from the data.

Step 6 — Train the Model

The model is trained using the training dataset.

During training, Linear Regression learns the relationship between the input features and the target variable by estimating the best-fitting coefficients.

Step 7 — Make Predictions

After training, the model generates predictions for both:

- Validation dataset (for evaluation)
- Test dataset (for final Kaggle submission)

Why two predictions?

Validation



Check Accuracy

Test



Final Kaggle Submission

Step 8 — Reverse Log Transformation

Since the target variable was transformed using `log1p()` before training, the predictions must be converted back to the original price scale using `expm1()`.

This ensures that the predicted values represent actual house prices.

Step 9 — Quick Look at Predictions

A comparison table is created to visually inspect the predicted values against the actual house prices.

This provides a quick understanding of the model's prediction quality before calculating evaluation metrics.



PART 10 — Model Evaluation

Step 1 — Import Evaluation Metrics

Several evaluation metrics are imported to measure the performance of the regression model.

Each metric evaluates the model from a different perspective, providing a more complete understanding of prediction quality.

Step 2 — Mean Absolute Error (MAE)

MAE measures the average absolute difference between the predicted values and the actual values.

Formula

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Lower MAE indicates better prediction accuracy.

Step 3 — Mean Squared Error (MSE)

MSE measures the average squared prediction error.

Formula

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Large errors receive much larger penalties because the errors are squared.

Step 4 — Root Mean Squared Error (RMSE)

RMSE is the square root of MSE.

Formula

$$RMSE = \sqrt{MSE}$$

Since RMSE has the same unit as the target variable (house prices), it is much easier to interpret.

Lower RMSE indicates a better model.

Step 5 — R² Score

The R² score measures how much of the variation in the target variable is explained by the model.

Formula

$$R^2 = 1 - \frac{SS_{Residual}}{SS_{Total}}$$

Step 6 — Print Everything Together

The evaluation metrics are displayed together to provide a concise summary of the baseline model's performance.

These values serve as a benchmark for comparison with more advanced regression models.

Step 7 — Cross Validation

Cross-validation evaluates the model on multiple train-validation splits instead of relying on a single split.

The dataset is divided into five folds.

Each fold is used once for validation while the remaining folds are used for training.

The final score is the average performance across all folds.

This provides a more reliable estimate of how well the model will perform on unseen data.

Step 8 — Interpretation

Suppose

Average CV RMSE = 24890

and

Validation RMSE = 24500

That's excellent.

It means

The model behaves consistently on unseen data.

If instead

Validation RMSE

18000

Cross Validation

42000

Then the model is likely **overfitting**.

Interview Question

Q: Why use Cross Validation instead of only Train-Test Split?

Answer:

A single train-test split may produce biased or unstable results depending on how the data is divided. Cross-validation evaluates the model

across multiple data splits, providing a more reliable estimate of its performance on unseen data and reducing the risk of overfitting to one particular split.

PART 11 — Advanced Regression Models & Model Comparison

Step 1 — Import Models

Several regression algorithms are imported to compare their predictive performance.

Each algorithm learns differently, allowing us to determine which model best captures the relationship between house features and sale prices.

Step 2 — Create Models

Each regression model is initialized with commonly used parameters.

For tree-based algorithms, a fixed `random_state` ensures reproducible results.

Why these models?

Model	Why use it?
Linear Regression	Baseline
Ridge	Controls overfitting
Lasso	Performs feature selection
ElasticNet	Combines Ridge + Lasso
Decision Tree	Learns nonlinear relationships
Random Forest	Reduces overfitting using many trees
Gradient Boosting	Learns from previous errors

Step 3 — Create Empty Results List

An empty list is created to store the evaluation metrics for every regression model.

This makes comparison easier later.

Step 4 — Train Every Model

Each regression model is trained using the same training data.

Predictions are generated on the validation dataset and evaluated using MAE, RMSE, and R^2 Score.

The results are stored for comparison.

Step 5 — Create Comparison Table

The evaluation metrics for all regression models are combined into a single table and sorted according to RMSE.

Lower RMSE indicates better predictive performance.

Step 6 — Best Model

The regression model with the lowest RMSE is selected as the current best-performing model.

This model will be further optimized using hyperparameter tuning.

Step 7 — Train Best Model on Full Training Data

The selected best-performing model is retrained using the training dataset so it can later be optimized through hyperparameter tuning.

Step 8 — Feature Importance (Tree Models Only)

For tree-based models, feature importance scores indicate which variables contributed most to the predictions.

This improves model interpretability and helps identify the most influential features.

Interview Question

Why compare multiple models instead of using Linear Regression directly?

Answer:

Different machine learning algorithms learn patterns differently. Linear Regression assumes a linear relationship, while tree-based methods can capture complex nonlinear interactions. Comparing multiple models helps identify the algorithm that provides the best predictive performance for the dataset.

PART 12 — Hyperparameter Tuning

Step 1 — Import Required Libraries

GridSearchCV is used to automatically search through different hyperparameter values and

select the one that provides the best cross-validation performance.

Step 2 — Create Parameter Grid

Since Lasso Regression has only one major hyperparameter (**alpha**), we define multiple possible values for Grid Search to evaluate.

Why Alpha?

Alpha controls the amount of regularization.

- Very small alpha → model behaves like Linear Regression.
- Very large alpha → many coefficients become zero.
- Best alpha → balances bias and variance.

Step 3 — Create Grid Search

GridSearchCV evaluates every alpha value using **5-fold Cross Validation**.

The alpha producing the lowest RMSE is automatically selected.

Why max_iter=10000?

Sometimes Lasso requires more iterations to converge.

Increasing max_iter prevents convergence warnings.

Why cv=5?

The training data is divided into 5 equal folds.

Each fold is used once as validation while the remaining four folds are used for training.

This provides a more reliable estimate of model performance.

Why n_jobs=-1?

All CPU cores are used to speed up computation.

Step 4 — Train Grid Search

Grid Search trains multiple Lasso models using different alpha values and evaluates each one using 5-fold cross-validation.

Step 5 — Best Alpha

The alpha value that achieved the best validation performance is selected automatically.

Step 6 — Best Cross Validation Score

This displays the best average cross-validation score obtained during Grid Search.

Since negative RMSE is used, values closer to zero indicate better performance.

Step 7 — Save Optimized Model

The optimized Lasso model returned by GridSearchCV is saved for evaluation and final prediction.

Step 8 — Predict Validation Data

The optimized model predicts on the validation set.

The inverse log transformation (`np.expm1`) converts predictions back to actual house prices.

Step 9 — Evaluate Optimized Model

The optimized model is evaluated using:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

These metrics are compared against the baseline model.

Step 10 — Compare Baseline vs Optimized

A comparison table is created to evaluate the improvement achieved through hyperparameter tuning.

A successful tuning process should generally result in:

- Lower RMSE
- Higher R^2 Score

Interview Question

Why was Lasso Regression selected instead of Random Forest?

Lasso Regression achieved the best validation performance among all tested models. Rather than choosing a complex model by default, the selection was based on objective evaluation metrics. This demonstrates a data-driven approach where the model is chosen because it generalizes best to unseen data.

Why does Lasso use only one hyperparameter?

Lasso Regression is a linear model with L1 regularization. Its primary tuning parameter is alpha, which controls the strength of regularization and determines how much the model shrinks feature coefficients. By adjusting alpha, Lasso can reduce overfitting and even perform automatic feature selection by setting some coefficients exactly to zero.

PART 13 — Final Model Training & Kaggle Submission

Interview Questions

Why train on the full dataset at the end?

Once the best model and hyperparameters have been selected, there is no longer a need

for a validation split. Training on the complete dataset allows the model to learn from all available examples, which can improve generalization on the unseen test set.

Why apply `np.expm1()` before submission?

The target variable was transformed using `np.log1p()` during training to reduce skewness and improve model performance. Predictions are therefore produced in the logarithmic scale and must be converted back to the original house price values using `np.expm1()` before submission.

Why must the submission contain only `Id` and `SalePrice`?

The Kaggle competition evaluates submissions using a predefined format. Including extra columns or incorrect column names will cause the submission to be rejected.